

TCS Stock Price Prediction - Interview Explanation

My project is a Stock Price Prediction System using Machine Learning. The objective of the project is to predict the closing price of TCS stock using historical stock market data and machine learning algorithms.

First, I imported the required libraries. I used Pandas and NumPy for data manipulation and numerical operations. I used Matplotlib for data visualization. From Scikit-Learn, I imported `train_test_split` for splitting the dataset into training and testing sets, `StandardScaler` for feature scaling, `LinearRegression` and `RandomForestRegressor` for machine learning models, and evaluation metrics such as Mean Absolute Error and R2 Score. I also imported Pickle to save the trained model.

Next, I loaded the TCS stock dataset using Pandas `read_csv()` and stored it in a DataFrame. I displayed the first five rows using `head()` to understand the data and printed the column names to see the available features.

After that, I selected only the important columns required for stock prediction. These columns were Date, Open, High, Low, Close, and Volume. This helped remove unnecessary data and focus only on relevant information.

Then I converted the Date column into a proper datetime format using `pd.to_datetime()`. By default, Pandas reads dates as strings, so this conversion allows Pandas to recognize and work with dates correctly.

Next, I performed feature engineering to create additional useful features from the existing data.

The first feature was the 10-day Moving Average, which is the average of the previous 10 closing prices. It helps identify short-term trends in the stock price.

The second feature was the 20-day Moving Average, which is the average of the previous 20 closing prices. It helps identify longer-term trends.

I then calculated Daily Return using percentage change. The formula is:

$$\text{Daily Return} = (\text{Today's Close} - \text{Yesterday's Close}) / \text{Yesterday's Close}$$

This measures the day-to-day percentage movement of the stock.

After that, I calculated Volatility. Volatility measures how much the stock price fluctuates over time. I calculated it as the rolling standard deviation of Daily Returns over a 10-day window.

A small standard deviation indicates that stock returns are relatively stable, while a large standard deviation indicates that stock prices fluctuate significantly and are more volatile.

Since Moving Averages and Volatility require previous data points, the first few rows contain missing values. For example, the first 9 rows cannot have a 10-day moving average and the first 19 rows cannot have a 20-day moving average. Therefore, I removed all rows containing null values using `dropna()`.

Next, I prepared the machine learning dataset.

I selected the input features as Open Price, High Price, Low Price, Volume, MA_10, MA_20, Daily Return, and Volatility.

The target variable was the Close Price because that is what I wanted to predict.

Before training the models, I performed feature scaling using StandardScaler.

The formula used by StandardScaler is:

$$z = (x - \text{mean}) / \text{standard deviation}$$

Feature scaling ensures that all features are on a similar scale, preventing features with large values, such as Volume, from dominating the model.

After scaling, I split the dataset into training and testing sets using `train_test_split()`.

I used 80% of the data for training and 20% for testing. The `random_state` value was fixed at 42 so that the results remain consistent every time the code is executed.

The first machine learning model I used was Linear Regression.

Linear Regression assumes a linear relationship between the input features and the target variable. Internally, it learns an equation of the form:

$$\text{Close Price} = b_0 + b_1x_1 + b_2x_2 + \dots + b_8x_8$$

Initially, the model is empty. During training, it learns the coefficients that minimize the prediction error between actual closing prices and predicted closing prices.

After training, I used the model to predict stock prices on the test data.

The second model I used was Random Forest Regressor.

Random Forest is an ensemble learning algorithm that consists of multiple decision trees. In my project, I created 100 decision trees using `n_estimators = 100`.

Each decision tree learns patterns from the training data. While building a tree, it evaluates different conditions such as `Open Price < 3550` and `Volume < 2200000` and chooses the condition that reduces prediction error the most.

The prediction error is typically based on the sum of squared differences between actual values and predicted values.

When a new input is provided, the sample passes through all 100 trees. Each tree generates its own prediction. The Random Forest then averages all these predictions to produce the final output. This averaging reduces overfitting and improves prediction accuracy.

After training both models, I evaluated their performance.

The first metric was Mean Absolute Error.

$$\text{MAE} = \sum | \text{Actual} - \text{Predicted} | / \text{Number of Samples}$$

This metric measures the average prediction error. Lower MAE indicates better performance.

The second metric was R² Score.

$$R^2 = 1 - [\sum (\text{Actual} - \text{Predicted})^2 / \sum (\text{Actual} - \text{Mean})^2]$$

R² measures how much of the variation in stock prices is explained by the model.

An R^2 value close to 1 indicates a strong predictive model, while a value close to 0 indicates poor predictive performance.

I printed the MAE and R^2 scores for both Linear Regression and Random Forest and compared their performance.

After evaluation, I saved the trained Random Forest model using Pickle. The model was stored in a file called model.pkl so that it can be reused later without retraining.

To visualize the results, I plotted a graph comparing actual stock prices and predicted stock prices.

The x-axis represents days and the y-axis represents stock prices.

This graph helps visually assess how closely the model's predictions match the actual stock prices.

Finally, I performed a sample prediction.

I created a sample input containing Open Price, High Price, Low Price, Volume, Moving Averages, Daily Return, and Volatility values.

Since the model was trained on scaled data, I first transformed the sample input using the same StandardScaler.

Then I passed the scaled input to the Random Forest model, which predicted the closing stock price.

The predicted closing price was printed as the final output.

Overall, this project helped me understand the complete machine learning workflow, including data preprocessing, feature engineering, feature scaling, model training, model evaluation, model persistence, and stock price prediction using machine learning.